CS260 Reading 5 (Week 5)

What I need to do

- read 10.1, 10.2, 10.3 (heaps)
- skim 10.5
- take notes
- answer the 2 questions
- upload notes + answers pic/pdf to gradescope

Notes

10.1 Data Structures: An Overview

- data structure = a way of organizing data + the operations it supports (arrays, linked lists, heaps, hash tables, search trees, etc.)
- choosing a DS is a design decision, just like choosing an algorithm - pick based on which operations you actually need fast
- **principle of parsimony**: use the leanest/simplest data structure that supports the operations you actually need, don't pay for extra generality you're not using

10.2 Supported Operations

- heap = container of objects, each with a key (priority), built for fast insert + fast extraction of the min (or max)
- core ops, both O(log n):
  - Insert(H, key) - add a new object
  - ExtractMin(H) - remove + return the object with the smallest key
- usually also: FindMin - O(1), peek at the min without removing it
- heaps are NOT good at: searching for an arbitrary key, deleting an arbitrary element (unless you keep extra bookkeeping/pointers)
- basically a specialized DS for "always give me the next smallest (or highest priority) thing, fast"

10.3 Applications

- **10.3.1 HeapSort**
  - insert all n elements into a heap, then ExtractMin n times in a row -> elements come out in sorted order
  - O(n log n) total; in-place version exists too
- **10.3.2 Event-driven simulation / scheduling**
  - keep pending events in a min-heap keyed by timestamp
  - ExtractMin to grab + process the next event, Insert to schedule new events as they're generated
  - way faster than scanning a whole list to find the next event each time
- **10.3.3 Median Maintenance**
  - goal: keep a running median of a stream of numbers, print it out whenever asked, without knowing in advance when that'll be
  - trick: keep 2 heaps - a max-heap for the lower half of numbers seen so far, a min-heap for the upper half
  - keep the two heaps balanced in size (differ by at most 1) - after each insert, rebalance by moving the top element from the bigger heap to the other
  - median = top of whichever heap has the extra element (or average of both tops if sizes are equal)
  - each insertion touches O(1) heap operations, each O(log i) at the i-th insertion

10.5 Implementation Details (skim)

- heaps are usually implemented as a **complete binary tree stored in an array** (no explicit pointers needed)
- for index i, parent/children are computable with simple arithmetic (e.g. children at 2i, 2i+1)
- heap property: every parent's key <= (or >=) both children's keys - weaker than a BST, only need local ordering, not global
- **Insert**: put new element in the next open leaf (end of array), then "bubble up" (sift-up) - swap with parent while heap property is violated -> O(log n) since tree height is O(log n)
- **ExtractMin**: swap root with the last element, remove the old root, then "bubble down" (sift-down) the new root, swapping with the smaller child until heap property restored -> O(log n)
- because the tree is always kept complete (filled left to right), height stays O(log n) automatically - no rebalancing logic needed like BSTs

Answers

Q1 (notes): see above - that's the reading notes.

Q2:

(a) How does the principle of parsimony apply to choosing a data structure?

- pick the simplest/leanest data structure that supports exactly the operations you need, nothing more
- heaps are a good example: if all you need is "insert stuff, always grab the min," you don't reach for a full BST or sorted array - those support way more (arbitrary search, order statistics, etc.) but cost more to build and maintain
- extra generality = extra cost in time and/or space you don't need to pay if a leaner structure gets the job done

(b) Total cost of maintaining the two heaps over n elements

- each insertion (the i-th one) costs O(log i): one Insert into one of the two heaps, plus possibly a rebalancing step (an Extract + Insert to move the top element across heaps), both O(log i)
- total cost = sum over i = 1 to n of O(log i) = O(log 1 + log 2 + … + log n) = O(log(n!)) = **O(n log n)**
- so even though any single insert/query at step i is only O(log i), maintaining the two heaps across all n insertions costs O(n log n) overall

(c) Decrease-key on a min-heap

- if you have a direct pointer/index to the node whose key is being lowered: update the value in place, then bubble it up (sift-up) - repeatedly compare to its parent and swap if it's now smaller, until the heap property is restored or it hits the root
- since a heap is a complete binary tree, height is O(log n), so bubble-up takes at most O(log n) swaps
- runtime: **O(log n)** - same as Insert/ExtractMin (assuming you already know the node's location; if you had to search for it first, that's an extra O(n) unless you maintain a key -> index map)